```
import heapq
from termcolor import colored
class PuzzleState:
    def __init__(self, board, parent, move, depth, cost):
        self.board = board
        self.parent = parent
        self.move = move
        self.depth = depth
        self.cost = cost

    def __lt__(self, other):
        return self.cost < other.cost
def print_board(board):
    print("+---+---+---+")
    for row in range(0, 9, 3):
        row_visual = "|"
        for tile in board[row:row + 3]:
            if tile == 0:
                row_visual += f" {colored(' ', 'cyan')} |"
            else:
                row_visual += f" {colored(str(tile), 'yellow')} |"
        print(row_visual)
        print("+---+---+---+")
goal_state = [1, 2, 3, 8,0,4,7,6,5]

moves = {
    'U': -3,
    'D': 3,
    'L': -1,
    'R': 1
}
def heuristic(board):
    distance = 0
    for i in range(9):
        if board[i] != 0:
            x1, y1 = divmod(i, 3)
            x2, y2 = divmod(board[i] - 1, 3)
            distance += abs(x1 - x2) + abs(y1 - y2)
    return distance
def move_tile(board, move, blank_pos):
    new_board = board[:]
    new_blank_pos = blank_pos + moves[move]
    new_board[blank_pos], new_board[new_blank_pos] = new_board[new_blank_pos], new_board[blank_pos]
    return new_board
def a_star(start_state):
    open_list = []
    closed_list = set()
    heapq.heappush(open_list, PuzzleState(start_state, None, None, 0, heuristic(start_state)))

    while open_list:
        current_state = heapq.heappop(open_list)

        if current_state.board == goal_state:
            return current_state

        closed_list.add(tuple(current_state.board))

        blank_pos = current_state.board.index(0)

        for move in moves:
            if move == 'U' and blank_pos < 3:
                continue
            if move == 'D' and blank_pos > 5:
                continue
            if move == 'L' and blank_pos % 3 == 0:
                continue
            if move == 'R' and blank_pos % 3 == 2:
                continue

            new_board = move_tile(current_state.board, move, blank_pos)

            if tuple(new_board) in closed_list:
                continue

            new_state = PuzzleState(new_board, current_state, move, current_state.depth + 1, current_state.depth + 1 + heuri
            heapq.heappush(open_list, new_state)

    return None

def print_solution(solution):
    path = []
```

```
    path = []
    current = solution
    while current:
        path.append(current)
        current = current.parent
    path.reverse()

    for step in path:
        print(f"Move: {step.move}")
        print_board(step.board)
initial_state = [2,8,3,1,6,4,7,0,5]
solution = a_star(initial_state)

if solution:
    print(colored("Solution found:", "green"))
    print_solution(solution)
else:
    print(colored("No solution exists.", "red"))
```

```
Solution found:
Move: None
+---+---+---+
| 2 | 8 | 3 |
+---+---+---+
| 1 | 6 | 4 |
+---+---+---+
| 7 |   | 5 |
+---+---+---+
Move: U
+---+---+---+
| 2 | 8 | 3 |
+---+---+---+
| 1 |   | 4 |
+---+---+---+
| 7 | 6 | 5 |
+---+---+---+
Move: U
+---+---+---+
| 2 |   | 3 |
+---+---+---+
| 1 | 8 | 4 |
+---+---+---+
| 7 | 6 | 5 |
+---+---+---+
Move: L
+---+---+---+
|   | 2 | 3 |
+---+---+---+
| 1 | 8 | 4 |
+---+---+---+
| 7 | 6 | 5 |
+---+---+---+
Move: D
+---+---+---+
| 1 | 2 | 3 |
+---+---+---+
|   | 8 | 4 |
+---+---+---+
| 7 | 6 | 5 |
+---+---+---+
Move: R
+---+---+---+
| 1 | 2 | 3 |
+---+---+---+
| 8 |   | 4 |
+---+---+---+
| 7 | 6 | 5 |
+---+---+---+
```